

Services/SystemAdminCoordinator.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4
5 namespace Week3
6 {
7     public class SystemAdminCoordinator : ICoordinatorRepository
8     {
9         private readonly List<Organization> _organizations = new();
10        private readonly List<Internship> _internships = new();
11        private readonly List<Student> _students = new();
12
13        public IReadOnlyList<Organization> Organizations => _organizations;
14        public IReadOnlyList<Internship> Internships => _internships;
15        public IReadOnlyList<Student> Students => _students;
16
17        public bool AddOrganization(Organization org)
18        {
19            if (_organizations.Any(o => string.Equals(o.Name, org.Name,
20StringComparison.OrdinalIgnoreCase)
21                && string.Equals(o.City, org.City,
22StringComparison.OrdinalIgnoreCase)))
23                return false;
24            _organizations.Add(org);
25            return true;
26        }
27
28        public IEnumerable<Organization> GetOrganizationsOverview() =>
29            _organizations.OrderBy(o => o.City).ThenBy(o => o.Name);
30
31        public void AddInternship(Internship internship)
32        {
33            _internships.Add(internship);
34        }
35
36        public IEnumerable<Internship> GetInternshipsOverview(Period period,
37InternshipCategory? category = null) =>
38            _internships.Where(i => i.Period.Year == period.Year &&
39i.Period.Semester == period.Semester
40                && (category == null || i.Category == category))
41                .OrderBy(i => i.OrganizationName);
42
43        public bool AssignStudentToInternship(Student student, string internshipId)
44        {
45            var internship = _internships.FirstOrDefault(i => i.Id ==
46internshipId);
47            if (internship == null) return false;
```

```
43
44         // rule: prevent same student (by id) assigned to multiple internships
in same period
45         if (_internships.Any(i => i.AssignedStudents.Any(s => s.StudentId ==
student.StudentId)
46             && i.Period.Year == internship.Period.Year &&
i.Period.Semester == internship.Period.Semester))
47             return false;
48
49         // if a student with same id exists in repository, use that instance
(preserve other data)
50         var existingStudent = _students.FirstOrDefault(s => s.StudentId ==
student.StudentId);
51         if (existingStudent != null)
52         {
53             // update name if provided
54             if (!string.IsNullOrEmpty(student.FirstName))
existingStudent.FirstName = student.FirstName;
55             if (!string.IsNullOrEmpty(student.LastName))
existingStudent.LastName = student.LastName;
56             student = existingStudent;
57         }
58         else
59         {
60             _students.Add(student);
61         }
62
63         // create a simple assignment object based on internship category and
add to internship
64         var assignment = CreateAssignmentFromCategory(internship.Category,
internship.Title);
65         internship.Assignments.Add(assignment);
66
67         internship.AssignedStudents.Add(student);
68         return true;
69     }
70
71     // helper to map category -> assignment subtype
72     private Assignment CreateAssignmentFromCategory(InternshipCategory
category, string title)
73     {
74         return category switch
75         {
76             InternshipCategory.Research => new ResearchAssignment(title),
77             InternshipCategory.Minor => new MinorAssignment(title, module:
"unknown"),
78             InternshipCategory.Engineering => new EngineeringAssignment(title),
79             _ => new ResearchAssignment(title)
80         };
81     }
```

```
82
83     // Withdraw internship (remove internship, unassign students and remove
internship contacts from org)
84     public bool WithdrawInternship(string internshipId)
85     {
86         var internship = _internships.FirstOrDefault(i => i.Id ==
internshipId);
87         if (internship == null) return false;
88
89         // unassign and remove students associated with this internship
90         foreach (var student in internship.AssignedStudents.ToList())
91         {
92             var existing = _students.FirstOrDefault(s => s.StudentId ==
student.StudentId);
93             if (existing != null) _students.Remove(existing);
94         }
95
96         // clear assignments for this internship
97         internship.Assignments.Clear();
98
99         // remove contact persons of this internship from the organisation (if
present)
100        var org = _organizations.FirstOrDefault(o => o.OrganizationId ==
internship.OrganizationId);
101        if (org != null)
102        {
103            foreach (var contact in internship.Contacts.ToList())
104            {
105                var match = org.Contacts.FirstOrDefault(c => c.Id ==
contact.Id);
106                if (match != null) org.Contacts.Remove(match);
107            }
108        }
109
110        _internships.Remove(internship);
111        return true;
112    }
113
114    // Mark internship completed with final grade
115    public bool MarkInternshipCompleted(string internshipId, double finalGrade)
116    {
117        var internship = _internships.FirstOrDefault(i => i.Id ==
internshipId);
118        if (internship == null) return false;
119        internship.MarkCompleted(finalGrade);
120        foreach (var assignment in internship.Assignments)
121        {
122            assignment.IsCompleted = true;
123        }
124        return true;
```

```
125     }
126
127     // Remove (unassign) a student from an internship (student remains in
student repo)
128     public bool RemoveStudentFromInternship(string studentId, string
internshipId)
129     {
130         var internship = _internships.FirstOrDefault(i => i.Id ==
internshipId);
131         if (internship == null) return false;
132         var student = internship.AssignedStudents.FirstOrDefault(s =>
s.StudentId == studentId);
133         if (student == null) return false;
134
135         internship.AssignedStudents.Remove(student);
136
137         return true;
138     }
139
140     // convenience queries
141     public Organization? GetOrganizationById(string organizationId) =>
_organizations.FirstOrDefault(o => o.OrganizationId == organizationId);
142
143     public IEnumerable<Internship> GetInternshipsByOrganizationId(string
organizationId) =>
144         _internships.Where(i => i.OrganizationId == organizationId).OrderBy(i
=> i.Period.Year).ThenBy(i => i.Period.Semester);
145
146     public IEnumerable<ContactPerson> GetContactPersonsForInternship(string
internshipId)
147     {
148         var internship = _internships.FirstOrDefault(i => i.Id ==
internshipId);
149         if (internship == null) return Enumerable.Empty<ContactPerson>();
150         return internship.Contacts.OrderBy(c => c.LastName).ThenBy(c =>
c.FirstName);
151     }
152
153     public bool WithdrawInternshipSimple(string internshipId) // kept for
backward compatibility
154     {
155         return WithdrawInternship(internshipId);
156     }
157
158     public bool AddStudent(Student student)
159     {
160         if (student == null || string.IsNullOrWhiteSpace(student.StudentId))
return false;
161         var existing = _students.FirstOrDefault(s => s.StudentId ==
student.StudentId);
```

```
163         if (existing != null)
164         {
165             if (!string.IsNullOrEmpty(student.FirstName))
existing.FirstName = student.FirstName;
166             if (!string.IsNullOrEmpty(student.LastName)) existing.LastName
= student.LastName;
167             return true;
168         }
169         _students.Add(student);
170         return true;
171     }
172
173     public bool AddContactPerson(string organizationId, ContactPerson contact)
174     {
175         if (contact == null || string.IsNullOrEmpty(organizationId))
return false;
176         var org = _organizations.FirstOrDefault(o => o.OrganizationId ==
organizationId);
177         if (org == null) return false;
178         if (string.IsNullOrEmpty(contact.Id)) contact.Id =
Guid.NewGuid().ToString();
179         // avoid exact duplicate entry by id
180         if (org.Contacts.Any(c => c.Id == contact.Id)) return true;
181         org.Contacts.Add(contact);
182         return true;
183     }
184
185     public IEnumerable<ContactPerson> GetContactPersonsByOrganization(string
organizationId)
186     {
187         var org = _organizations.FirstOrDefault(o => o.OrganizationId ==
organizationId);
188         if (org == null) return Enumerable.Empty<ContactPerson>();
189         return org.Contacts;
190     }
191 }
192 }
```